WEB-250 | Debugging Setup — Visual Studio Code + Xdebug

Course: WEB-250 Database-Driven Website
Points: 100 — separate Moodle assignment
Due: As posted in Moodle
Folder: `web250/debugging_setup`
Software: VS Code, PHP 8.1 or newer, MAMP (Mac) or XAMPP (Windows), Xdebug 3
Starter files: `debugging_setup_student_starter.zip`

> **Two assignments, two grades:** This is **Debugging Setup (100 points)**. The other assignment is **asgn08 — Active Record Directory (100 points)**. Submit each in its **own Moodle activity**. Nothing in this debugging rubric is part of the Active Record rubric.
>
> **Good news:** You do **not** need MySQL, the `wnc_birds` database, API keys, or an `.env` file for this exercise. We will debug small PHP programs first and use the same tools later when our applications become more complex.

1. 📖 Introduction — Why learn a debugger?

You have probably debugged PHP by adding `print`, `var_dump()`, or `die()` to your code. Those are helpful, but they can get messy. A **debugger** lets you pause a program while it is running and **see what PHP knows at that exact moment**.

With Xdebug connected to Visual Studio Code (VS Code), you can:

- 🔴 Set a **breakpoint**: stop before a particular line runs.
- 👀 Inspect **variables**: check a value without adding a `print` statement.
- 👣 **Step Over (F10)**: run the current line, but do not open a function called by that line.
- ➡️ **Step Into (F11)**: move inside a function you are calling.
- ↩️ **Step Out (Shift+F11)**: finish the current function and return to its caller.
- ▶️ **Continue (F5)**: run until the next breakpoint or the end of the request.
- 🕵️ Use **Watch** to follow the value of an expression as you step.
- 🧭 Read the **Call Stack** to see which functions led to the current line.
- 🎯 Set a **conditional breakpoint** to stop only when a condition is true.

Think of the debugger as a way to ask, *"What is the value here, and how did execution reach this line?"*

How the pieces connect

```
Browser -> MAMP/XAMPP Apache -> PHP + Xdebug
                                  |
                                  | debugging connection (port 9003)
                                  v
                            VS Code + PHP Debug extension
                                  |
                                  v
```

```
                        breakpoint, Variables, Watch,
                        Call Stack, Step Over/Into/Out
```

Important: Our class uses Apache browser port **55000** and Xdebug-to-VS Code port **9003**. They are two different connections. The browser opens `http://localhost:55000/...`, while Xdebug connects to the VS Code listener at `127.0.0.1:9003`. If your instructor has you use a different Apache port, change the **browser URL only**; do not add `:55000` to `xdebug.client_host`.

2. 🧪 Guided Sandbox — Set up the debugger and try it (not graded separately)

2.1 Create the folder and examine the files

Create the new folder **alongside** `asgn08`, not inside it. Do not rename it `asgn08/debugging`.

Mac (MAMP with its standard document root):

```
/Applications/MAMP/htdocs/web250/debugging_setup
```

Windows (XAMPP with its standard document root):

```
C:\xampp\htdocs\web250\debugging_setup
```

If your MAMP/XAMPP document root is different, place `web250/debugging_setup` under **your actual working document root**. Keep your existing `asgn07` and `asgn08` folders.

Unzip **`debugging_setup_student_starter.zip`** **into** **`debugging_setup`**. You should end up with:

```
web250/
  asgn07/                       <- existing project
  asgn08/                       <- separate 100-point assignment
  debugging_setup/              <- this 100-point assignment
    .vscode/
      launch.json               <- supplied VS Code listener setup
    sandbox/
      sandbox_breakpoints.php   <- complete working example
    exercises/
      exercise01_variables.php
      exercise02_loop_watch.php
      exercise03_functions_stack.php
      exercise04_find_bug.php
    evidence/
      .gitkeep                  <- add screenshots here
    index.php                   <- simple links to the programs
    .gitignore                  <- excludes local diagnostics
```

```
README.md                    <- complete the observations
DEBUGGING_SETUP_START_HERE.md
```

Open **File > Open Folder** in VS Code and choose the **`debugging_setup` folder itself**. This matters: VS Code should find `.vscode/launch.json` immediately under the folder you opened. No terminal-only configuration is required.

2.2 Install the PHP Debug extension in VS Code (Mac and Windows)

1. Open VS Code.
2. Open **Extensions** (the four-square icon), or press **Cmd+Shift+X** on Mac / **Ctrl+Shift+X** on Windows.
3. Search for **PHP Debug** and install the extension published by **Xdebug** (`xdebug.php-debug`).
4. Keep PHP Intelephense if you already use it. Intelephense helps with code editing; **PHP Debug** supplies the VS Code connection to the running Xdebug extension.
5. If VS Code requests **Trust** for the course folder, trust only the folder that you recognize and created.

Installing a VS Code extension **does not install Xdebug into PHP**. We have to configure both pieces.

2.3 Find the PHP configuration used by your **browser**

Create this temporary file directly inside `debugging_setup`:

File: `_local_phpinfo.php`

```php
<?php
phpinfo();
```

Open it through **Apache**, not by double-clicking the PHP file:

Your setup	Example browser URL
Mac — MAMP, class Apache port 55000	`http://localhost:55000/web250/debugging_setup/_local_phpinfo.php`
Windows — XAMPP, class Apache port 55000	`http://localhost:55000/web250/debugging_setup/_local_phpinfo.php`

For this course, use port 55000 on Mac and Windows. If your working Apache port differs, substitute that port in the browser address. Do not accidentally open port 8888 if a different application (such as Jupyter) is listening there. If the page downloads instead of displaying PHP information, fix Apache/PHP before continuing.

On the `phpinfo()` page, locate and record **PHP Version** and **Loaded Configuration File**. Search the page for **Xdebug**. If Xdebug is already installed, find its **version** and settings. Use these facts to edit the **correct PHP configuration**, not an unrelated copy of PHP.

> 🔐 `phpinfo()` can reveal details about your computer and environment. **Never upload this file to your webhost or GitHub.** The supplied `.gitignore` excludes it. Delete `_local_phpinfo.php` after setup. You are *not* asked to submit a screenshot of the phpinfo page.

2.4 🍎 Mac — Enable Xdebug in MAMP

These steps assume regular **MAMP** on macOS. If you use **MAMP PRO**, read the PRO note below.

1. In MAMP, note the PHP version selected for Apache. Stop the servers.
2. Open the **Loaded Configuration File** reported by the browser's `phpinfo()` page. With the usual MAMP setup, this resembles:

   ```
   /Applications/MAMP/bin/php/phpX.Y.Z/conf/php.ini
   ```

   Replace `X.Y.Z` with the actual installed version. Do **not** type the placeholder literally.
3. Search that file for `xdebug` or `zend_extension`. If a compatible Xdebug extension is already supplied but commented out, enable **that existing entry**. **Do not invent an extension filename or paste a path from another student's computer.** Avoid adding a second `zend_extension` entry when one is already active.
4. Once Xdebug **version 3** is loaded, make sure these values are present **once** in the active configuration (adjust existing lines instead of accumulating duplicates):

   ```
   xdebug.mode=debug
   xdebug.start_with_request=yes
   xdebug.client_host=127.0.0.1
   xdebug.client_port=9003
   ```
5. Save the configuration, start the MAMP servers, and reload `_local_phpinfo.php`. Verify the **Xdebug** section appears and **Step Debugging is enabled**.

MAMP PRO: Enable the Xdebug extension in the MAMP PRO PHP settings if necessary, and edit the **PHP template** using **File > Open Template > PHP (php.ini)** for the selected PHP version. MAMP PRO can regenerate its `php.ini` when servers restart, so directly editing the generated file may not work. Restart and check the browser's `phpinfo()` again.

If Xdebug is missing entirely: Do not download a random `.so` file or change your system/Homebrew PHP to fix MAMP. Consult the Xdebug installation wizard, which works from your PHP configuration details, or ask your instructor for the correct MAMP build. The extension must match the PHP version Apache is actually running.

2.5 🍎 Mac — Increase MAMP's FastCGI timeout **before debugging**

This step is for regular MAMP on Mac, not Windows/XAMPP. It solves a problem you might otherwise encounter while learning to step through PHP: MAMP can run PHP using **FastCGI**, which may stop waiting after **30 seconds**. When you pause at a VS Code breakpoint and spend time examining variables, Apache can time out and show **Internal Server Error (HTTP 500)** in the browser. That does **not** necessarily mean your PHP code is broken.

We will give PHP **600 seconds (10 minutes)** to remain paused while we investigate. This is a **local development setting**; you do not edit your GitHub repository or your public webhost to make this change.

1. **Stop the servers in MAMP.** Make a backup of the Apache configuration before changing it. In the Mac Terminal, run:

   ```
   cp /Applications/MAMP/conf/apache/httpd.conf
   ~/Desktop/httpd.conf.before-xdebug
   ```

   This saves a backup on your Desktop. If macOS denies access, ask your instructor for help rather than changing file permissions broadly.

2. Open this file in **VS Code**:

   ```
   /Applications/MAMP/conf/apache/httpd.conf
   ```

   Search with **Cmd+F** for `FastCgiServer`. If you cannot find that directive, **do not add a new one**. Your MAMP installation might be configured differently; ask your instructor.

3. Find the existing line for MAMP's PHP FastCGI server. It will look like this:

   Before:

   ```
   FastCgiServer /Applications/MAMP/fcgi-bin/php.fcgi -socket
   httpdFastCGI.sock
   ```

   Add `-idle-timeout 600` to the end of that **same line**:

   After:

   ```
   FastCgiServer /Applications/MAMP/fcgi-bin/php.fcgi -socket
   httpdFastCGI.sock -idle-timeout 600
   ```

   If the line already has an `-idle-timeout` option, **change its existing value** rather than adding a second one. Keep the rest of the line exactly as it was.

4. **Save `httpd.conf` and restart MAMP's servers.** If Apache does not start, restore the backed-up file and ask your instructor to check the configuration. Do not continue until MAMP is running normally.

5. **Verify the fix after setting up the VS Code listener in Section 2.7.** Set a breakpoint in `sandbox/sandbox_breakpoints.php`, start listening in VS Code, and open the sandbox in your browser. **Stay in VS Code while it is paused.** Wait **45 seconds**, then press **F10** (Step Over) and **F5** (Continue). Only then return to the browser. It should finish with:

   ```
   Bird: Carolina Wren
   Total sightings: 5
   ```

   If you see this output rather than an HTTP 500 error, the 30-second timeout problem is resolved.

If an HTTP 500 still appears: Look in the Apache error log immediately after reproducing it:

```
tail -n 30 /Applications/MAMP/logs/apache_error.log
```

A message containing `FastCGI ... idle timeout (30 sec)` indicates the 30-second timeout is still active. Confirm that you edited the active configuration and restarted MAMP. An HTTP 500 with **different** error messages needs a different fix; do not assume every 500 is a timeout.

MAMP PRO: Do not edit the ordinary MAMP `httpd.conf` blindly. MAMP PRO can generate its Apache configuration from templates and settings; consult the PRO-specific configuration workflow or your instructor. **Windows/XAMPP students skip this Mac-only section.**

> **Why this matters:** Pausing for 45 seconds to inspect a call stack or Watch value is a normal part of debugging. A web server timeout should not force you to rush. Later, when working on your hosted website, you will **not** enable Xdebug or copy these local server settings to the host.

2.6 Windows — Enable Xdebug in XAMPP

1. Open the **XAMPP Control Panel**. Stop Apache.

2. Choose **Apache > Config > PHP (php.ini)**, or open the file shown as **Loaded Configuration File** on the browser `phpinfo()` page. A common path is:

   ```
   C:\xampp\php\php.ini
   ```

3. Search for `xdebug` and `zend_extension`. If Xdebug 3 is already installed and active, keep its working `zend_extension` line and skip downloading anything.

4. **If Xdebug is missing:** Open xdebug.org/wizard and follow the instructions for **your specific PHP build**. Windows Xdebug DLLs must match PHP version, architecture, thread safety, and compiler

details. The wizard will provide the correct DLL and installation directions. Then add or enable the **exact** `zend_extension` entry for the DLL you installed. Never guess the filename.

5. In the active `php.ini`, set these Xdebug **version 3** options:

```
xdebug.mode=debug
xdebug.start_with_request=yes
xdebug.client_host=127.0.0.1
xdebug.client_port=9003
```

6. Save, **restart Apache**, and reload `_local_phpinfo.php`. Confirm that **Xdebug** is loaded and **Step Debugging is enabled**.

Note for either operating system: These are **Xdebug 3** settings. Older tutorials often use Xdebug 2 settings such as `xdebug.remote_enable` and port 9000. Do **not** mix those old settings with Xdebug 3. If an earlier lab used debugger port **9001**, it must match in both PHP and `.vscode/launch.json`. For this lab, standardize on **9003** wherever possible.

2.7 Connect VS Code to Xdebug

The starter ZIP includes the file `.vscode/launch.json`:

```
{
  "version": "0.2.0",
  "configurations": [
    {
      "name": "Listen for Xdebug (local browser)",
      "type": "php",
      "request": "launch",
      "hostname": "127.0.0.1",
      "port": 9003
    }
  ]
}
```

What it means: `type: "php"` uses the PHP Debug extension; `request: "launch"` starts a VS Code **listener**, not Apache; `hostname: "127.0.0.1"` keeps the listener on your own machine; `port: 9003` must match `xdebug.client_port`. Because PHP and VS Code both run on your own computer using the same files, you do **not** need remote `pathMappings`.

1. Start MAMP or XAMPP and confirm Apache is running.
2. In VS Code, open `sandbox/sandbox_breakpoints.php`.
3. Click in the gutter **to the left of** the line `$total = $today + $yesterday;` to make a **red breakpoint dot**.

4. Open **Run and Debug** (play-and-bug icon). Choose **Listen for Xdebug (local browser)** and click the green triangle (or press **F5**). Leave it listening.

5. In your **web browser**, visit:

   Mac:
   `http://localhost:55000/web250/debugging_setup/sandbox/sandbox_breakpoints.php`
   Windows:
   `http://localhost:55000/web250/debugging_setup/sandbox/sandbox_breakpoints.php`
   (If your Apache server uses a different port, change only the port in the browser URL.)

6. VS Code should **pause on that line**. The current line is highlighted; the browser will be waiting for PHP to finish. **That is expected. Do not refresh the browser or expect to see the final output while paused.**

7. Expand the **Variables** section in Run and Debug. Hover over `$today` and `$yesterday` in the editor. Add `$total` to the **Watch** panel if desired.

8. Click **Step Over (F10)** once. The assignment line runs, so `$total` becomes **5**. Step to a new statement to inspect the updated value.

9. **Mac/MAMP timeout test:** If you completed Section 2.5, pause for at least **45 seconds** at a breakpoint during this test. You should still be able to step and continue normally.

10. Click **Continue (F5)** to let PHP finish **before** checking the browser. The browser should display:

```
Bird: Carolina Wren
Total sightings: 5
```

11. Stop the VS Code debugger (the stop-square icon) once the request is complete. **Delete `_local_phpinfo.php`** now that setup is verified.

Checkpoint: If pressing F5 immediately works but waiting longer than 30 seconds while paused causes HTTP 500, check **Section 2.5** (Mac/MAMP FastCGI timeout) and the Apache error log. Do not change `xdebug.client_host` to include port 55000.

If a function key changes your volume instead of debugging, use the buttons on the VS Code debug toolbar; some laptops need the **Fn** key in combination with F5/F10/F11.

2.8 Troubleshooting: Breakpoints, timeouts, and HTTP 500 errors

Work through this checklist in order — change **one thing at a time**:

Symptom	What to check
Browser page loads, but VS Code never pauses	Did you start **Listen for Xdebug** before opening the browser URL? Is the breakpoint on executable PHP code?

Symptom	What to check
Xdebug is absent in `phpinfo()`	Enable/install the Xdebug extension in the **web PHP** configuration, restart Apache, and refresh.
Xdebug appears, but step debugging is disabled	Confirm `xdebug.mode=debug` and restart Apache. `xdebug.start_with_request=yes` starts the session automatically for this lab.
VS Code listens on one port but Xdebug uses another	Make `xdebug.client_port` and `launch.json` agree (recommended **9003**). Apache's port is unrelated.
Breakpoint stays unverified / does not bind	Open the real folder under `htdocs` in VS Code, not a second copy elsewhere. Reload the same localhost URL.
`php -v` looks different from `phpinfo()`	The terminal may be using another PHP installation. For **browser-based debugging**, use the PHP version and configuration from browser `phpinfo()`.
Windows reports it cannot load an Xdebug DLL	Revisit the Xdebug wizard. The DLL must match the installed PHP build exactly.
Browser seems to hang while VS Code is paused	The browser is waiting for the paused PHP request. Stay in VS Code until you press **Continue (F5)**, then return to the browser.
Mac/MAMP displays **Internal Server Error (500)** after you spend roughly 30 seconds inspecting variables	Check the Apache error log for `FastCGI ... idle timeout (30 sec)`. Follow **Section 2.5** to append `-idle-timeout 600` to MAMP's existing `FastCgiServer` line. Save and restart MAMP.
Mac/MAMP still shows HTTP 500 despite the timeout change	Look at `/Applications/MAMP/logs/apache_error.log`. Confirm that the edited Apache configuration was loaded. Different errors require a different diagnosis.
Debugging started on port 9001 in an earlier lab	Either align **both** configurations to 9001 or change **both** to this assignment's standard 9003.

For more advanced problems, VS Code's Debug Console can show adapter messages; the Xdebug extension also documents optional local logging. Do not put debugging logs in a public website folder.

3. 🔬 Experiment — Predict, change, run (not graded)

These are short experiments with the **complete sandbox**, not extra submission exercises. Write down a **prediction first**, run the test, then restore the original working code. Keep the sandbox folder as a personal reference library.

Experiment A — Before vs. after

Move the red breakpoint to the `print` statement **after** the line that calculates `$total`.

- **Predict:** Will `$total` be `0` or `5` when VS Code pauses?
- **Run:** Reload the sandbox browser URL while VS Code listens.
- **Observe:** The debugger stops **before** its highlighted line executes.
- **Restore:** Return the breakpoint to the calculation line, or remove it.

Experiment B — Try different data

Temporarily change `$today` from `2` to `7`.

- **Predict:** What number should the page print?
- **Watch:** Pause before the `$total` calculation, then step over and inspect the value.
- **Restore:** Return `$today` to `2`.

Experiment C — Remove a breakpoint

Click the red dot to remove the breakpoint and reload the page.

- **Predict:** Will VS Code pause? Does the script still work?
- **Explain:** Why is a breakpoint a better choice than inserting a `die()` during this kind of investigation?

4. ✅ Checkpoints — What should you see?

Checkpoint	Expected observation or question
Browser test	The sandbox prints `Total sightings: 5` on its original data.
Before calculating	`$today` is `2`, `$yesterday` is `3`, and `$total` has not yet been assigned the result.
After Step Over	`$total` is `5`.
Mac/MAMP pause test	After waiting 45 seconds at a breakpoint (with the 600-second FastCGI timeout configured), Step Over and Continue still work; no HTTP 500 appears.
Continue	PHP finishes the request and the browser displays its output.
Variables vs. Watch	Variables shows in-scope values; Watch follows expressions you choose.
Step Over vs. Step Into	Over executes the call without entering it; Into pauses inside the called function.
Call Stack	How can you tell which function called the function you are currently inside?
Conditional breakpoint	Why might stopping at `dayIndex === 2` be more useful than pausing on every loop iteration?

Before starting graded work: Confirm that your local VS Code listener actually pauses at the sandbox breakpoint. If it does not, fix the setup first rather than guessing about the code in the next section.

5. 💻 Independent Assignment — Four debugging exercises (graded, **100 points total**)

The four programs are already included in the starter ZIP. You **do not** need to invent large programs or write SQL. Your work is to **use** the debugger, record evidence of what you observed, and make one small code correction.

Rules for the exercises: Use the **browser + VS Code/Xdebug debugger**, not only `print()` or `var_dump()`. Take your screenshots *while paused* at meaningful breakpoints. The supplied code is simple on purpose; the challenge is learning the debugging tools. In `README.md`, record your **prediction**, **observed value**, and **what you learned** for each exercise. You may add comments, but preserve the exercise filenames.

Exercise 01 — 🔴 Breakpoints, Variables, and Watch (15 points)

File: `exercises/exercise01_variables.php`

1. Read the script without running it. Predict the value of `$observations` **before** and **after** its `+= 2` statement.
2. Put a breakpoint on the `$observations += 2;` line.
3. Start **Listen for Xdebug**; load the exercise from your localhost browser.
4. While the breakpoint is paused, inspect `$observations` in **Variables**, then add `$observations` to **Watch**. Record its current value.
5. Click **Step Over** once and record the new value.
6. Click **Continue** and verify the final browser output.
7. Save `evidence/01_variables.png` showing VS Code **paused**, a breakpoint, and the **Variables** / **Watch** values. (A `.jpg` is also acceptable.)

Expected browser output:

```
Bird: Carolina Wren
Habitat: Thickets
Sightings: 5
```

README questions: What did the breakpoint stop *before* executing? What changed after Step Over?

Exercise 02 — 🕵️ Loop tracing and a conditional breakpoint (15 points)

File: `exercises/exercise02_loop_watch.php`

1. Inspect the array of daily sightings. **Predict** the final total and average.
2. Put a normal breakpoint on the line `$total += $count;` inside the loop. Add `$total` to **Watch**.
3. Refresh the URL, step through at least two iterations, and record how `$total` changes.
4. Remove the normal breakpoint. **Right-click the gutter** on that same line and choose **Add Conditional Breakpoint** (or edit the breakpoint condition).
5. Enter this PHP expression as the condition:

```
$dayIndex === 2
```

6. Refresh while the debugger is listening. The debugger should pause when the **third** array entry is about to be added (array indices start at zero). Look at `$dayIndex`, `$count`, and `$total` before the line runs.
7. Take `evidence/02_conditional.png` showing the conditional pause and the visible variable values. Continue and check the completed output.

Expected browser output:

```
Total sightings: 10
Average per day: 2.50
```

Checkpoint: At the conditional pause, expect `$dayIndex` = **2**, `$count` = **1**, and `$total` = **6** (the third count has **not** been added yet).

README questions: What did `$total` equal at the first two pauses? Why did the conditional breakpoint save time?

Exercise 03 — ➡️ Step Into, Step Out, and Call Stack (15 points)

File: `exercises/exercise03_functions_stack.php`

1. Before running, predict the final text from the program.
2. Put a breakpoint on the line that calls `describeBird('Eastern Towhee', 2)`.
3. Start the listener and reload the page. Choose **Step Into (F11)** to enter `describeBird()`.
4. Advance to the call to `formatObservation()`. Use **Step Into** again.
5. While you are **inside `formatObservation()`**, expand the **Call Stack** panel. You should see `formatObservation`, then `describeBird`, followed by the main PHP script (display may use slightly different labels).
6. Save `evidence/03_call_stack.png` showing the paused function and the **Call Stack**.
7. Use **Step Out** to return to `describeBird()`, then **Step Out** again to return to the main script. Continue to finish.

Expected browser output:

```
Eastern Towhee: 2 sightings
```

README questions: Which function called `formatObservation()`? Why is **Step Into** more useful than Step Over here?

Exercise 04 — 🪲 Find and repair a logic error (15 points)

File: `exercises/exercise04_find_bug.php`

This program **runs without a PHP error**, but its result is wrong. The data are `[2, 3, 4]` and the expected total is **9**.

1. **Before modifying the code**, run the page normally (or Continue through the debugger) and record the wrong output.
2. Place a breakpoint on the line that changes `$total` in the loop. Add `$total` to **Watch**.
3. Step through **each** iteration. For each one, write down `$count` and `$total` **before and after** that statement executes. Identify the line causing the error.
4. Make the **smallest possible code correction**. Add a brief comment explaining the original mistake; do not rewrite the entire function or replace the loop with `array_sum()`.
5. Run the fixed script again. It must print:

   ```
   Birds counted: 9
   ```

6. For the screenshot `evidence/04_fixed_bug.png`, put a conditional breakpoint on the corrected statement with condition `$count === 4`. Reload. **Before** the third value is added, `$total` should now be **5**. Capture the paused screen, then Continue and verify that the browser prints **9**.

README questions: Why did the original program complete without an exception? What did Watch reveal that a single final `print` could not?

5.1 📸 Evidence and README

Complete the supplied `README.md` template. Include:

- Your operating system (**Mac/MAMP** or **Windows/XAMPP**), browser PHP version, Xdebug version, which `php.ini` you used, and debug listener port. On **Mac/MAMP**, confirm whether the FastCGI timeout was set to **600 seconds** and whether the **45-second pause test** succeeded. Windows students record **N/A** for this setting. **Never include passwords or environment dumps.**
- A short entry for **each exercise** with your prediction, observed result, and explanation of the debugger feature used.
- The original wrong result and your one-line fix for Exercise 04.
- **Four screenshots** in the `evidence/` folder, with short captions in the README:
  - `01_variables.png` — paused breakpoint and Variables/Watch.
  - `02_conditional.png` — paused conditional breakpoint in the loop.
  - `03_call_stack.png` — paused inside a function with Call Stack visible.
  - `04_fixed_bug.png` — paused in the corrected loop, with Watch showing the running total.

You may use `.jpg` instead, but keep the numbered filenames recognizable. Crop screenshots to the relevant VS Code window; don't show personal account information. Your `README.md` and images should make it possible for the instructor to verify that you **used the debugger**, not just ran the scripts.

5.2 🌿 GitHub and the webhost

This is a **separate course folder**, not a Git repository inside asgn08:

1. Keep `debugging_setup/` in your **existing course Git repository**, alongside `asgn08/`. Do not initialize an extra `.git` in the debugging folder.
2. Commit the starter code, your corrected Exercise 04, `.vscode/launch.json`, README, and screenshots. If the repository's older `.gitignore` excludes every `.vscode/` directory, make an exception for **`debugging_setup/.vscode/launch.json`**; that listener configuration belongs in this assignment. Do not commit Xdebug logs, diagnostic `phpinfo()` pages, or other local-only files.
3. You can do this assignment on `main` before starting the asgn08 `dev-active-record` branch. That particular branch name is **reserved for asgn08** and is not a requirement for debugging setup. If you use a separate branch for debugging setup, merge it into `main` before submitting.
4. Push your work to GitHub. Verify that the `debugging_setup` folder, `README.md`, `.vscode/launch.json`, exercises, and screenshots are visible on **`main`**.
5. On the webhost, create `web250/debugging_setup/`. Upload **only the demonstration PHP pages**: `index.php`, `sandbox/`, and `exercises/`. Your revised `exercise04_find_bug.php` must print the correct **9**. Update your existing **web250 index** to link to `debugging_setup/index.php`, while preserving earlier assignment links.
6. **Do not upload** `.vscode/`, `evidence/`, `README.md`, `.gitignore`, `_local_phpinfo.php`, logs, or any local PHP/Xdebug configuration. **Do not attempt to activate Xdebug on your public webhost.** Debugging sessions and screenshots are performed locally; the hosted pages merely demonstrate that the PHP scripts work.

AI and the index pages: "You can use AI to write the index files because it is grunt work." You may use AI for navigation/index links and to explain errors, but you must personally use VS Code's debugger for the screenshots and observations, and make the Exercise 04 correction yourself.

5.3 📫 Submit in Moodle — *Debugging Setup* activity

Paste **both** links in the **Comments** section of the separate **Debugging Setup** Moodle assignment:

```
GitHub (main): https://github.com/YOUR-USERNAME/YOUR-
REPOSITORY/tree/main/debugging_setup
Webhost demo: https://YOUR-DOMAIN/web250/debugging_setup/index.php
```

Your repository layout may use a different root; link directly to the debugging_setup folder on `main`. The hosted page must load without an instructor login. **Your screenshots and observations are submitted through GitHub**, not published on the webhost. Do not confuse these two URLs with the URLs you submit separately for asgn08.

✅ Final checklist

- ☐ `web250/debugging_setup` is a sibling of asgn08, not inside it.
- ☐ PHP Debug is installed in VS Code; browser PHP has Xdebug **3** with step debugging enabled.
- ☐ `launch.json` uses the same Xdebug port (recommended **9003**).
- ☐ Browser examples use the class Apache port **55000** (or my confirmed alternate port); Xdebug listens on **9003**.
- ☐ On Mac/MAMP, I backed up `httpd.conf`, set the existing `FastCgiServer` line to `-idle-timeout 600`, restarted MAMP, and passed the 45-second pause test. (Windows: N/A.)

- [ ] Breakpoints actually stop the PHP code when visiting a **localhost** browser URL.
- [ ] All four exercises work, and Exercise 04 has been repaired to print **9**.
- [ ] README contains predictions, observations, and explanations for all four exercises.
- [ ] Four debugger screenshots are in the GitHub `evidence/` folder.
- [ ] PHP demo files work on your webhost, but Xdebug/VS Code settings and diagnostic files are **not** published.
- [ ] The web250 index links to `debugging_setup/index.php`.
- [ ] GitHub and webhost URLs are in the **Debugging Setup** Moodle Comments.

💯 Debugging Setup rubric — **100 points**

Category	What the instructor checks	Points
Debugger installation and configuration	Correct browser-side PHP/Xdebug setup, working VS Code PHP Debug extension, matching listener port, actual paused execution; Mac/MAMP timeout adjustment and successful 45-second pause test (Windows: not applicable).	**20**
Exercise 01 — breakpoints and variables	Prediction, correct before/after values, Variables and Watch, Step Over, verified output.	**15**
Exercise 02 — loop and conditional breakpoint	Correct total/average, tracked loop values, conditional breakpoint stops on the expected iteration, verified output.	**15**
Exercise 03 — functions and Call Stack	Demonstrates Step Into and Step Out, identifies callers correctly, uses Call Stack, verifies output.	**15**
Exercise 04 — locate and fix a bug	Captures incorrect behavior, traces with debugger, makes minimal correct fix, verifies 9, explains the problem.	**15**
README and screenshots	Four relevant paused debugger screenshots plus clear predictions, observations, explanations, and local setup details.	**10**
GitHub, webhost, and course index	All required files on GitHub `main`, only safe PHP pages hosted, hosted links work, course index updated.	**8**
Moodle submission	Both correct GitHub and webhost links in the separate Moodle Comments.	**2**
TOTAL		**100**

📚 References (optional help)

- VS Code: Debug code — Breakpoints, Watch, Variables, Call Stack, and stepping controls.
- PHP Debug extension for VS Code — Installation and listener configuration.
- Xdebug 3: Step debugging — Modes, request activation, hosts, and ports.
- Xdebug installation wizard — Identify the correct version or DLL for PHP.
- MAMP: Edit PHP configuration — Find the active MAMP PHP configuration.

- Apache mod_fastcgi documentation — Reference for the `-idle-timeout` server option.
- MAMP PRO: PHP template editing — Why PRO configuration needs a template.

Remember: you will use these debugger skills again in asgn08 and future WEB-250 assignments. Keep this folder as part of your personal reference library.